

研究ストーリー

S2620154 吉田 将衛

2026 年 5 月 10 日

一言で言う（elevator pitch）

代替が存在するにもかかわらず残存するメソッド（価値なしコードの一形態）を、@Deprecated を proxy ground truth として用い、コード構造・使用パターン・Git 履歴から早期に検出する手法を提案する（動機: 価値なしコードの唯一の公式ラベルである @Deprecated ですら付与まで平均 8 年のラグが存在し、ラベルなし事例が大量に存在する）

確定済みスコープ

項目	決定内容	根拠・備考
エコシステム	Java	Sawant et al. の先行研究群に準拠
ターゲットベニュー	SCAM	ソースコード解析・MSR 寄りの貢献が評価される
ステークホルダー	提供者 (API producer) 向け	Sawant 2019 「提供者ポリシーが消費者行動を動かす」と整合
フレーム	手法提案 (@Deprecated を proxy GT とした代替存在メソッドの早期検出)	Javadoc に書いてあれば開発者はすでに知っている → proxy GT が届く前にドキュメント非依存シグナルで検出する

1 背景

価値のないコードとソフトウェア保守負債

ソフトウェアの進化に伴い、より優れた代替が追加された後も旧機能が削除されずに残存するケースが生じる。本研究ではこのような「代替が存在するにもかかわらず残存するメソッド」を価値のないコードの一形態として捉える。そのようなコードがコードベースに残り続けると、技術的負債として蓄積される **[TODO: 根拠: 価値なしコードの残存が保守コストを増大させることを示す先行研究 (技術的負債・dead code 系)]**。技術的負債の議論では、問題が定量的に可視化されることで初めて優先付けの対象となる **[TODO: 引用が必要 (技術的負債の可視化と優先付けに関する先行研究)]**。すなわち、どのメソッドが「価値なし」であるかを把握するには、客観的なラベルが必要である。

@Deprecated を proxy ground truth として

前節で述べた「代替が存在するにもかかわらず残存するメソッド」を検出するには、どのメソッドが「代替あり」であるかを示すラベルが必要である。

Java の @Deprecated アノテーションは API 提供者が「このメソッドはもう使うべきでない」と宣言する公式の仕組みであり、非推奨化の理由は多様である。Sawant et al. [4] の調査 (Java 4 ライブラリ・374 件) では 12 分類が識別されており、そのうち最多 (37%、138 件) が「代替機能の導入 (NF: New Feature Introduced)」すなわち、より良い代替 B が追加されたため旧機能 A が不要となったケースである。NF カテゴリは定義的に「代替あり」を宣言するものであり、本研究が対象とする「代替が存在するメソッド」に直接対応する。したがって、**NF 型の @Deprecated が本研究の proxy ground truth である**。本研究はこのカテゴリに対象を絞る。

ただし、この proxy はノイジーである：代替が存在するにもかかわらず @Deprecated が付与されないメソッドが大量に存在する (偽陰性)。これは次節で示す 8 年ラグの本質であり、proxy GT なしの状態で価値なしコードが長期間放置されるという問題の核心でもある。

Proxy Ground Truth 自体の遅延問題

代替機能が追加された後も旧機能への @Deprecated 付与が長期間遅延するケースが広く存在することが、本研究のパイロット分析で示唆される (詳細は次節)。

Brito et al. [1] が破壊的変更を導入した開発者 56 名に行ったインタビューでは、@Deprecated を付与しなかった 17 名への追加質問において、「**非推奨化後のメンテナンスコスト増加 (44%)**」が最多回答であった。しかし本研究のパイロット分析では、代替追加から @Deprecated 付与までの平均ラグが **8 年 (2,904 日)** であることが観察されており (次節)、先送りによって @Deprecated という proxy GT ラベルが届かないまま価値なしコードが長期間放置されることを示す。

さらに、Sawant [2] は提供者の非推奨化ポリシー (削除積極性等) が消費者の移行行動を直接左右することを大規模分析 (25,357 プロジェクト) で実証した。提供者が proxy GT の付与を後回しにすることで、消費者の移行も連鎖的に遅延するという構造的な問題が生じる。

なぜ Proxy Label を待たずに検出するか

平均 8 年のラグは、代替が追加された後も proxy GT ラベルが届かないまま価値なしコードがラベルなし状態で放置されることを意味する。したがって、@Deprecated の付与を待つだけでは価値なしコードを体系的に把握することはできない。

根本的な困難がある：**B が A の代替であるという関係は、提供者が @Deprecated を付与する時点で初めて宣言されるものである**。付与前の段階では B が追加されていてもその代替関係はコード上の事実として確定しておらず、Javadoc に代替が明記されているのはパイロット分析でも **15.9%** に過ぎない (詳細は次節)。

先行研究 (Sawant [4, 3, 2]) はいずれも付与済みの @Deprecated を起点とするため、「付与前の代替存在メソッドの発見」はスコープ外であった。したがって、proxy GT ラベルを待たずに代替存在メソッドを検出するには、コード構造・使用パターン変化・Git 履歴という客観的シグナルから**代替関係を推定する**アプローチが必要となる。本研究は OSS の Git 履歴を用いた遡及的分析により、観測可能なシグナルが将来の @Deprecated 付与と相関するかを検証する。

2 Motivation: Proxy Ground Truth としての @Deprecated の特性分析

(2026-05-04 計測済み)

指標	値	補足
has_replacement_pct	15.9%	@Deprecated 全件中 Javadoc に代替メソッド記載あり
alt_found_pct	93.9%	記載ありのうち Git から代替を自動発見できた割合
nf_first_pct	12.6%	@Deprecated 全件中「代替が先に追加」のケース
nf_first / has_replacement	≈ 79%	Javadoc 代替記載ありの約8割が NF 先
lead_time_mean (NF 先のみ)	2,904 日 (約8年)	代替追加 → @Deprecated 付与までの平均ラグ

→ **観察**: Javadoc に代替が明記されているケース (15.9%) ですら proxy GT ラベルが8年放置される。残り 84.1% は Javadoc にも記載がなく、代替が存在するにもかかわらず proxy GT ラベルなし状態のまま放置されているメソッドが大量に存在することを示唆する。

3 アプローチ (提案手法)

3段階で構成する。まず「各メソッドの代替候補 B を推定する (Stage 1)」、次に「代替が存在するメソッドをランク付けする (Stage 2 → RQ1)」、最後に「誰を先に対処するか (Stage 3 → RQ2)」を問う。

Stage 1: シグナル抽出 @Deprecated を proxy ground truth として用いるため、Javadoc を一切用いずに各メソッド A に対して代替候補 B を推定するシグナルを計算する。以下のシグナルを個別・組み合わせで試す (どれが効くかは RQ1 で明らかにする)。

- **型互換性**: 戻り値型が一致し引数型が互換 (代替の必要条件)。ただし同クラス内で型互換なペアは膨大になるため、絞り込みとしては弱い
- **命名パターン**: `getX/getXNew` などの規則的ペア。精度は高いが、易しいケース (おそらく Javadoc にも書かれている) しか取れない可能性
- **コード構造類似**: 同クラス内の AST 距離・シグネチャ類似度。ただし代替は実装が変わることも多く、類似度の定義 (AST / トークン / 埋め込み) によって捕まえるものが全く異なる
- **コミット共変**: B 追加コミットで A の caller が同時変更
- **caller 段階移行**: A の caller が時間とともに B へ移っていく Git 履歴
- **LLM ゼロショット**: 上記シグナルが効かない場合の対抗手法として比較

Javadoc は答え合わせ専用 (学習・推論では使わない)。

Stage 2: 代替存在メソッドの候補ランキング Stage 1 のシグナルを用いて、未付与メソッドの中から「代替が存在するメソッド (= 価値なしコード)」をランク付けする。`@Deprecated` 付与済みの NF 型事例を proxy GT (正例) として、各 T_{obs} における識別精度を Recall@ k および MRR で評価する。出力は提供者向けの候補リスト。

Stage 3: 優先順位付け Stage 2 で候補が得られたとして、「どれを先に対処すべきか」を問う。右打ち切りを伴う時間問題のため、生存分析 (Cox モデル等) を軸に検討する。出力は提供者向けのトリアージリスト。

4 Research Questions

ギャップから RQ への導出

§ 背景で示した通り、本研究のギャップは「代替が存在するにもかかわらず残存するメソッドを定量的に発見する手段がない」ことにある。`@Deprecated` はその価値なしコードの最良の proxy ground truth だが、NF 型ですら付与まで平均 8 年のラグが存在し、大量のラベルなし事例が生じている。この困難の中核は、 B が A の代替であるという関係はコード上の事実として直接観測できず、提供者が `@Deprecated` を付与する時点で初めて宣言されるという点である。パイロット分析が示すように、Javadoc に代替が明記されているのは全体の 15.9% に過ぎず、残り 84.1% については代替関係が文書化されていない。

そこで本研究は、proxy GT ラベルを待たずに代替存在メソッドを検出するために、Javadoc に頼らずコード構造・使用パターン・Git 履歴から読み取れる手がかり (以下、コードと履歴に残る手がかり) を用いて、段階的に二つの問いに答える。

両 RQ 共通の時間軸 本研究では、ある観測時点 T_{obs} におけるプロジェクト開始から T_{obs} までのコード状態と Git 履歴のみを手法の入力とする。すなわち手法は T_{obs} より未来の情報を一切参照しない。RQ1 では T_{obs} を変数として掃引し、リードタイム ($T_{\text{dep}} - T_{\text{obs}}$) と検出精度の関係を測る。RQ2 では T_{obs} を最新コミットに固定して実プロジェクトに適用する。手法そのものは両者で共通であり、差は T_{obs} の置き方と答え合わせが可能か否かのみである (表 1)。

段階	RQ1 (リードタイム特性の検証)	RQ2 (実プロジェクト適用)
観測時点 T_{obs}	各事例ごとに $[T_B + \epsilon, T_{\text{dep}} - \epsilon]$ を掃引。 T_B は代替 B の追加時点、 T_{dep} は <code>@Deprecated</code> 付与時点	リポジトリの最新コミット時点
特徴の観測窓	プロジェクト開始 $\sim T_{\text{obs}}$	プロジェクト開始 $\sim T_{\text{obs}}$
手法適用時点	各 T_{obs} (A はまだ未付与)	T_{obs} (未付与メソッド全体が対象)
答え合わせ	T_{dep} 時点の Javadoc 記載 B と照合	右打ち切りデータに対する生存分析 (Cox 比例ハザード) で C-index・時間依存 AUC を評価

Table 1: RQ1/RQ2 の時間軸

4.1 RQ1: コードと履歴から観測できるシグナルは、代替が存在するメソッドを `@Deprecated` 付与のどれだけ前から検出できるか？

時間軸を一級の変数として扱う理由 本研究の動機は「代替 B の追加から `@Deprecated` (proxy GT) 付与までの平均ラグが約 8 年」という事実にある。仮に手法が付与直前 $T_{\text{dep}} - \epsilon$

の状態では機能しないなら、それは proxy GT ラベルが届く直前を確認しているに過ぎず、早期検出には貢献しない。逆に B の追加直後 $T_B + \epsilon$ で機能すれば、代替が存在するメソッドを **8 年前から** 発見できることが原理的に可能となる。そこで RQ1 では観測時点 T_{obs} を $[T_B + \epsilon, T_{\text{dep}} - \epsilon]$ の範囲で掃引し、**リードタイム vs 検出精度の特性曲線**を求めることを目的とする。

母集団・入力・予測対象

- **母集団**: Java OSS プロジェクト群 **[TODO: プロジェクト選定基準と件数を § 評価設計 で定義]** から収集した事例 $(A, B, T_B, T_{\text{dep}})$ の集合。 A は @Deprecated 付与済みかつ Javadoc に代替メソッドが明記されたメソッド、 B はその代替メソッド、 T_B は B がコミットに初出現した時点、 T_{dep} は A に @Deprecated が付与された時点。パイロットでは全 @Deprecated の 14.9% ($= 15.9\% \times 93.9\%$) がこの条件を満たす
- **各事例ごとの観測時点群**: T_{obs} を以下の複数点で評価する。
 - $T_{\text{dep}} - \epsilon$ (付与直前。最も観測情報が豊富)
 - $T_{\text{dep}} - 1$ 年, $T_{\text{dep}} - 3$ 年, $T_{\text{dep}} - 5$ 年 (ラグの中盤)
 - $T_B + \epsilon$ (代替が現れた直後。理論値最速)
- **入力**: 各 T_{obs} について、プロジェクト開始から T_{obs} までのコード状態と Git 履歴のみ
- **予測対象**: 各 T_{obs} において、同一プロジェクト内のメソッド集合の中から、 T_{dep} 時点で Javadoc が指す代替メソッド B ($=$ 代替が存在することの証拠) を 1 件特定する。これにより対象 A が「代替が存在するメソッド (価値なし)」であることが確認される
- **答え合わせ**: Javadoc 記載の B と推定結果の一致を Recall@ k / MRR で評価。Javadoc 自体は学習・推論に一切用いず、proxy GT の正解確認専用とする

評価指標 RQ1 はランキングタスクである (正解の代替 B は各事例につき 1 件)。正解が 1 件のとき Recall@ $k = \text{Hit Rate}@k$ (上位 k 件に B が入るか否かの 0/1) であり、Precision@ $k = \text{Recall}@k / k$ と等価なため Precision は省略する。

- **Recall@ k (主指標, $k \in \{1, 3, 5\}$)**: 上位 k 件に正解 B が含まれるか。 $k = 5$ を主指標とする根拠は、提供者が実際に確認できるショートリストの上限が実務上 5 件程度という前提、およびコード推薦研究での慣例 (@1/@5/@10) に従うことにある。
- **MRR (補助指標)**: $\text{MRR} = \frac{1}{|D|} \sum \frac{1}{\text{rank}(B)}$ 。 Recall@ k が「入ったか否か」の 0/1 しか測れないのに対し、MRR は 2 位と 50 位を区別するため、リードタイムが伸びたときのランキング悪化の粒度を細かく追える。
- **リードタイム vs 精度曲線**: 各 T_{obs} における Recall@5 と MRR をプロットし、付与直前から T_B 直後へ遡るにつれて精度がどう悪化するかを可視化。
- **実用閾値**: ある精度水準 (例: Recall@5 $\geq 80\%$) を維持できる最大リードタイム ($=$ 「8 年中、何年前まで遡れるか」)

ベースラインと比較設計 比較群を役割の異なる 3 層に分けて設計する。

層 1—下限値の確定

- **B0: ランダム選択。** 同一プロジェクト内のメソッドから無作為に B を 1 件選ぶ。提案手法が「シグナルなし」の基準を超えることを確認する最低限の参照点 (Recall@1 $\approx 1/|M|$ となるべき)。

層 2—アブレーション (シグナル別寄与の分離)

提案手法は命名・コード構造・Git 履歴を統合して用いる。各シグナルがリードタイム別にいつ立ち上がるかを分離するために、一つずつ有効にした単独モデルを設定する。

- **B1: 命名規則のみ。** $getX/getXNew \cdot doY/doYV2$ 等の正規表現ペアで候補を生成。 T_B 直後から利用可能な最もコストの低いシグナル
- **B2: コード構造類似のみ。** AST 距離・シグネチャ類似度のみ。Git 履歴を一切参照しない
- **B3: Git 履歴のみ。** caller 段階移行・コミット共変のみ。AST 解析を行わない

B1~B3 のリードタイム vs 精度曲線を重ねることで、「どのシグナルがいつ有効になるか」を定量化する。(仮説: 命名は T_B 直後から有効・Git 移行シグナルは累積に時間を要する)

層 3—対抗手法 (LLM との対比)

- **B4: LLM ゼロショット抽出。** Javadoc を除去したメソッド本体と同一プロジェクト内の候補リストを LLM に与え、代替を 1 件選ばせる。

B4 はアブレーション (シグナル分離) ではなく、「LLM だけで十分では？」という代替アプローチとの対比である。この比較が答える問いは、「コードと履歴から計算したシグナルが LLM の暗黙知識を超える独自寄与を持つか」であり、とくに T_{obs} が T_B 直後 (Git 履歴が薄い時点) と T_{dep} 直前 (情報が豊富な時点) で逆転が起きるかが焦点となる。プロンプト設計・モデルバージョン・API コストは § 評価設計で記述する。

すべての比較群を同一の T_{obs} 掃引で評価し、リードタイム vs 精度曲線を重ねて描く。

スコープ境界と反証可能性 本手法はすべての代替関係を当ててることを目指さない。コード構造・使用パターン・Git 履歴のいずれかに痕跡を残すサブセットのみを対象とする。

反証可能な主張: 本研究の中核的主張は「ランダム選択 (B0) を有意に上回るリードタイム範囲が少なくとも 1 つ存在する」ことである。この主張は以下の形で反証される:

- $T_{obs} = T_{dep} - \epsilon$ (情報が最も豊富な時点) においてすら Recall@5 が B0 水準を上回らない場合 → 観測可能なシグナルに代替関係の情報は含まれないという否定的結論を報告する
- 有意に上回る範囲が存在しても、その最大リードタイムが 1 年未満にとどまる場合 → 「8 年のラグを縮める」という実用価値は限定的であることを認める

「リードタイムが長いほど精度が下がる」傾向自体は否定せず、悪化の度合いと、実用的な閾値 (例: Recall@5 $\geq 80\%$ を維持できる最大リードタイム) を定量化することを RQ1 の貢献とする。さらに、当てられた事例と外した事例の差を分析することで「本手法が機能する条件」 (例: caller の段階的移行が観測できる場合・命名が規則的ペアの場合 等) を明示し、適用可能な文脈を提供者に示す。

4.2 RQ2: RQ1 で検出された代替が存在するメソッドを共変量とする生存分析モデルは、その対処優先順位をどの程度の精度でランク付けできるか？

RQ1 から RQ2 への設計上の連鎖 RQ1 が答えるのは「どのメソッド A が代替が存在するメソッド（価値なし）か」という検出の問いである。しかし提供者が実際に必要とするのは「プロジェクト内の検出済み候補の中で、どれを先に対処すべきか」という優先順位付けである。前者は代替の存在を明らかにするが、複数の候補が存在するとき**どれが急を要するか**は答えない。ここで「対処の緊急度」の代理指標として @Deprecated 付与のハザードを用いる。

この問いは本質的に時間の問題であり、二値分類や回帰では扱えない。未付与メソッドのうち将来付与されるものは**右打ち切り**されているが、これはまさに生存分析 (survival analysis) が設計された問題設定である。そこで RQ2 は RQ1 で同定された代替候補を共変量として取り込み、**時間依存共変量を含む拡張 Cox 比例ハザードモデル**でハザード順ランキングを提供者に提供する。

RQ1 への依存リスクと頑健性: RQ2 の問題設定 (§ 生存分析の問題設定) は「RQ1 の手法で代替候補 B が同定された全 A 」を観察対象とするため、RQ1 の再現率が低ければ RQ2 の対象母集団も縮小する。しかし RQ2 のモデルは RQ1 の出力を**共変量の一つ**として扱うに過ぎず、命名・AST 距離・Git 履歴等の共変量は RQ1 とは独立に計算できる。ベースライン S4 (RQ1 結果を共変量に含めない Cox) との差分が「RQ1 の出力が RQ2 のランキング精度に与える上乗せ分」を定量化し、両 RQ の連鎖が実用的に成立することを裏付ける。

生存分析の問題設定

- **観察対象 (subject)** : RQ1 の手法で代替候補 B が同定された全 A
- **時間原点 $t = 0$:** T_B (代替メソッド B が初出現したコミット時点)
- **イベント:** A に @Deprecated が付与される
- **観察時間:**
 - イベント発生例: $T_{\text{dep}} - T_B$
 - 右打ち切り例: $T_{\text{now}} - T_B$
- **時間不変共変量:** 命名規則ペアか否か、 A と B の AST 距離・シグネチャ類似度 等
- **時間依存共変量:** A の caller のうち B へ移行した割合、コミット共変累積回数、 B の caller 数の伸び率 等
- **モデル:** 拡張 Cox 比例ハザードモデル

母集団

- **イベント発生群:** Javadoc に代替が明記され @Deprecated が付与された 14.9% の事例 (T_{dep} が観測される)
- **右打ち切り群:** 最新スナップショット時点で未付与のメソッドのうち、RQ1 の手法で代替候補 B が同定されたもの (T_{now} で打ち切り)

評価指標 RQ2 が答える問いは「ハザード順位の精度」と「予測確率の信頼性」の両面を含む。5 指標を役割別に 3 群に分けて設定する。

群 1—ランキング精度 (RQ2 の中核)

- **Concordance index (C-index)** : 無作為に選んだ 2 メソッドのうち先に非推奨化される方を正しくランクできる確率。提供者向けトリアージリストの精度に直結するため主指標とする (0.5 = ランダム、1.0 = 完全予測)
- **時間依存 AUROC (1 年・3 年・5 年)** : 「 t 年以内に付与されるか」という各地平線での二値識別能力。C-index が全時間軸を一括評価するのに対し、提供者が計画する具体的な期間 (次期リリースサイクル等) で識別できるかを測る。地平線ごとに性能プロファイルが異なれば、「短期予測は得意・長期は苦手」という実用上の限界として報告する

群 2—キャリブレーション (ランキングを確率として使えるか)

- **Brier スコア**: $\hat{S}(t|x)$ (予測生存確率) と実測値の平均二乗誤差。C-index はランキング順位しか評価せず、「ハザード 0.8」が本当に 80% の確率を意味するかは測れない。§ 応用で上位 N 件を選ぶ際の閾値設定を信頼できるものにするために必要。また S2 (ロジスティック回帰、右打ち切り無視) との Brier 差が「打ち切りを正しく扱うことでキャリブレーションが改善するか」を示す

群 3—可視化と解釈 (結果の説明可能性)

- **Kaplan-Meier 曲線**: 評価指標ではなく可視化ツール。パイロットの「平均 8 年ラグ」を単一数値ではなく生存関数の分布として描き、「ハザードの高い群 vs 低い群」の層別 KM 曲線でモデルが実際に分離できているかを視覚的に確認する
- **ハザード比**: 各共変量の効果量を表す解釈ツール。(例: 「caller 移行率が 50% 進むとハザード X 倍」) 提供者が「どのシグナルが最も非推奨化を促進するか」を理解するための Actionable Insight として提示する

ベースラインと比較設計 RQ2 の核心は「なぜ生存分析でなければならないか」と「なぜ時間依存共変量が必要か」を定量的に示すことにある。そのため、評価対象を提案手法 (時間依存共変量を含む拡張 Cox) と以下のベースラインで対比する。

- **S0: ランダムランキング**。下限値の確定 (C-index ≈ 0.5 となるべき)
- **S1: メソッド年齢ランキング**。「古い順に並べれば良いのでは?」という素朴反論への防御。パイロットの 8 年ラグから自然に導かれる仮説
- **S2: ロジスティック回帰 (主ベースライン)**。右打ち切りを単に「未付与 = 負例」として扱い、全共変量を投入。S2 と提案手法の C-index・Brier 差が、生存分析採用の必要性そのものを表す
- **S3: 時間依存共変量を除外した Cox (時間不変共変量のみ)**。S3 と提案手法の差が動的シグナル (caller 移行率等) の寄与を表す
- **S4: RQ1 結果を共変量に含めない Cox**。段階的設計 (RQ1 $\rightarrow$ RQ2) の連鎖が実用ランキングに寄与することを裏付ける

主張ラインは二段: S2 との差で「打ち切りを正しく扱う必要性」、S3 との差で「時間依存共変量の必要性」を順に示し、両者が同時に効いて初めて**拡張 Cox**という選択が正当化されるという論理にする。

応用: 提供者向けランキング 学習済みモデルを最新スナップショット時点に適用し、未付与メソッドを**現時点でのハザード**でランク付けする。上位 N 件は提供者がトリアージすべき候補となる。サンプル定性検証（コミット履歴・Issue・PR を手動確認）により、「妥当な候補」「誤検出」「議論済みで意図的に放置」を分類する。

スコープ境界・前提

- **選択バイアス**: イベントを生むのは過去に @Deprecated を付与した経験のある提供者のみ。一切付与しないプロジェクトは無限打ち切り扱いとなり、ベースラインハザードを下方に歪める可能性がある
- **意図的先送りの扱い**: Brito et al. [1] が報告した「コスト懸念で意図的に未付与」は競合リスク (competing risk) として扱う余地があるが、永久放置の判定は原理的に不可能なため本研究では単一イベント Cox に留める
- **時間原点の定義**: T_B を「 B の最初のコミット」とするが、テスト未整備な初期版を含む可能性があるため § 評価設計で頑健性を検証する **[TODO: T_B の定義の頑健性検証方針を § 評価設計 で詳述]**
- **比例ハザード仮定**: caller 移行効果は時間と共に変化し得るため、Schoenfeld 残差検定で仮定の妥当性を検証する

5 期待される貢献

- ✓ @Deprecated (proxy GT) が NF 型ですら平均 8 年遅延するという定量的実証 (= ラベルなし事例が大量に存在する証拠, パイロット済み)
- @Deprecated を proxy ground truth として用いた代替存在メソッド早期検出フレームワークの提案
- 複数 Java OSS プロジェクトにわたる大規模定量分析 (RQ1)
- ドキュメント非依存のシグナル設計 (コード構造・使用パターン・Git) とアブレーション (命名/構造/履歴/LLM ゼロショット) による**シグナル別寄与のリードタイム特性**の定量化 (RQ1)
- 上記シグナルと生存分析を組み合わせた代替存在メソッドの対処優先順位付け手法の提案と、打ち切り無視ロジスティック・時間依存共変量なし Cox との比較による有効性検証 (RQ2)

6 差別化・位置づけ

軸	先行研究	本研究
対象	非推奨化後の消費者行動 (Sawant 2019, Cogo 2021)	代替が存在するにもかかわらず 未検出のまま残存 するメソッドの発見
シグナル	@Deprecated 解析 / 手動分析	ドキュメント非依存 (コード・使用・Git)
貢献	非推奨化の分類・影響測定	proxy GT の 8 年遅延実証 + proxy GT を待たない早期検出フレーム
ステークホルダー	消費者 (移行コスト)	提供者 (価値なしコードの定量的可視化)

7 ストーリーの変遷

バージョン	日付	主な変更
v1	2026-04-28	初稿（雛形作成）
v2	2026-04-28	問題設定の甘いポイント整理・ステークホルダーを提供者向けに確定・スコープ（Java/SCAM）確定
v3	2026-04-28	パイロット実証の結果パターン別シナリオ（A～H）を追記
v4	2026-05-04	パイロット実証数値取得。フレームを“prediction / early detection” から “Deprecation Debt Discovery” ヘピボット。
v5	2026-05-04	手法提案論文として再設計。Javadoc 依存の RQ2 を廃止。ドキュメント非依存シグナル（コード・使用・Git）による候補検出を手法の核心に。
v6	2026-05-10	フレームを「非推奨化遅延」から「価値なしコード（代替存在メソッド）の検出」ヘピボット。 @Deprecated を proxy ground truth として明示定義。RQ 全面再設計（タスクは維持・意味付けを刷新）。背景を 4 節構成に再構成。

References

- [1] Aline Brito, Laerte Xavier, Andre Hora, and Marco Tulio Valente. Why and how Java developers break APIs. In *Proceedings of the 24th International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 2017.
- [2] Anand Ashok Sawant. *The impact of API evolution on API consumers and how this can be affected by API producers and language designers*. PhD thesis, Delft University of Technology, 2019.
- [3] Anand Ashok Sawant, Maurício Aniche, Arie van Deursen, and Alberto Bacchelli. Understanding developers’ needs on deprecation as a language feature. In *Proceedings of the 40th International Conference on Software Engineering (ICSE)*, pp. 561–571. ACM, 2018.
- [4] Anand Ashok Sawant, Guangzhe Huang, Gabriel Vilen, Stefan Stojkovski, and Alberto Bacchelli. Why are features deprecated? An investigation into the motivation behind deprecation. In *Proceedings of the 34th IEEE International Conference on Software Maintenance and Evolution (ICSME)*, pp. 13–24. IEEE, 2018.